



colegio
CALASANZ
SALAMANCA

Paseo de Canalejas, 139-159
37001 Salamanca
Tel.: 923 26 79 61 | Fax: 923 27 04 54
www.calasanzsalamanca.es
CENTRO PRIVADO CONCERTADO

AL PUNTO

CFGS DAW

CURSO 2025/2026

Héctor Corrales Pérez

Diego Ruiz García

Sergio Salinas Sierra

COLEGIO CALASANZ SALAMANCA

Índice

1.	Introducción	3
2.	Descripción de la aplicación.	4
3.	Plan de empresa.....	5
4.	Tecnologías escogidas y justificación	7
5.	Diseño de la aplicación.....	8
5.1.	Diagramas y definición de casos de uso.....	8
5.2.	Diagramas de clase.....	15
5.3.	Modelo entidad relación	16
6.	Arquitectura de la aplicación	17
6.1.	Estructura del proyecto.....	17
6.2.	Librerías externas utilizadas.....	19
7.	Manual de despliegue	19

1. Introducción

En un sector con aproximadamente 300.000 locales es complicado lograr centrarse en lo verdaderamente importante, los clientes, centro de nuestra actividad deben sentirse comprendidos, escuchados y acompañados incluso antes de cruzar las puertas del restaurante. Al Punto es una herramienta que piensa en ellos, en ofrecer al usuario una experiencia única que le anime a quedarse, que marque la diferencia frente a locales que no han comprendido que el cliente es el centro, la piedra angular de nuestro sector.

En un panorama donde estamos rodeados de diferentes negocios de hostelería, Al Punto proporciona ese toque mágico que anima al cliente a volver al local. Esto lo hace mediante un sistema de ofertas, estos recibirán retornos en puntos que darán acceso a un marketplace en el cual podrán ser usados; además de una sección de reseñas, porque la opinión de nuestros clientes es fundamental para nosotros y por ello les proporcionamos las herramientas necesarias para que los clientes se comuniquen de una forma abierta con el local en cuestión.

Al Punto es una aplicación preparada para todos aquellos restaurantes que quieran una diferenciación, ofrecemos una amplia gama de funcionalidades entre ellas un landing page de presentación del restaurante con un diseño personalizado y adaptado al gusto del cliente, en la landing page entre otras cosas podremos encontrar un menú que hará la navegación simple e intuitiva donde aparte de cierta información del local en cuestión podremos encontrar una carta interactiva con los productos del restaurante, sus reseñas y la puntuación que otros usuarios le han dado. En el menú se mostrará un botón de login donde si el usuario no tiene cuenta se podrá registrar proporcionando datos como nombre, apellido, email, teléfono y contraseña.

Tras haberse registrado se podrá acceder a un panel personal donde se podrá consultar la información personal. En este panel habrá varias opciones, entre ellas se encuentran reservas, historial, pedidos, tarjeta de fidelización, marketplace, cartera de puntos, tickets y reseñas.

2. Descripción de la aplicación.

Al Punto es una plataforma SaaS (*Software as a Service*) de gestión de fidelización y operativa para restaurantes. Aunque cada local tiene su identidad visual (Landing Page), el sistema centraliza la lógica de usuarios, reservas y puntos de fidelización en una arquitectura unificada.

2.1. Arquitectura de usuarios

Para el correcto funcionamiento de la lógica de negocio, se definen tres tipos de perfiles:

- **Ciente:** usuarios finales que gestionan sus reservas, pedidos y puntos. Además, pueden subir los tickets de su pedido para acumular puntos canjeables en el marketplace y escribir reseñas de sus consumiciones.
- **Empleado:** con acceso a todas las opciones del cliente, incluyendo la gestión de estos.
- **Administrador:** puede realizar todas las funcionalidades del cliente y del empleado, además de la creación, modificación y eliminación de mesas.

2.2. Flujo de pantallas y funcionalidad detallada

A. Landing page del restaurante

- **Página “Home”:** presentación visual, acceso al menú, al registro o inicio de sesión, sección de historia y una recomendación de platos variable.
- **Menú interactivo:** listado de platos divididos por categorías. Permite ver las reseñas de cada plato.
- **Módulo de reservas:** formulario donde el usuario elige fecha, número de comensales, mesa y hora. El sistema valida en tiempo real la disponibilidad de mesas contra la base de datos de SQLite.

B. Zona personal

Al iniciar sesión, el cliente accede a una SPA (*Single Page Application*) con:

- **Zona personal:** visualización de la tarjeta de fidelización digital (un código QR de identificación), saldo acumulado donde los puntos se generan automáticamente tras cada consumición pagada y un gráfico de estadísticas que incluye reservas, tickets y puntos.
- **Historial:** registro de reservas, pedidos y ofertas del marketplace canjeadas.
- **Marketplace:** tienda interna donde el usuario puede canjear sus puntos por beneficios.
- **Tus comentarios:** se muestran las reseñas ya escritas y aparece la opción de añadir una a los platos ya consumidos.
- **Realizar pedido:** interfaz de carrito de compra para seleccionar platos a consumir.
- **Tickets:** dividido en dos secciones. Por un lado, se podrá ver los tickets subidos y, por el otro, la propia opción de subida.
- **Administración:** teniendo los permisos adecuados, se podrá gestionar las acciones del cliente y gestionar las mesas del restaurante.

3. Plan de empresa

Debilidades	Amenazas	Fortalezas	Oportunidades
Complejidad: El sistema se compone de una gran cantidad de funcionalidades lo que dificulta su desarrollo, validación y la adopción por usuarios no técnicos	Resistencia a la digitalización, se sigue prefiriendo métodos manuales o TPVs simples. Además, de que puede generar sensibilidad por el precio.	Necesidad real: El sector de la hostelería en España representa aproximadamente un 6.7% del PIB Español y la gestión urgente es crucial especialmente para las PYmes	Digitalización acelerada del sector: Muchos restaurantes están buscando soluciones tecnológicas post-pandemia. El momento es propicio.
Dependencia del ser humano: Muchas de las funcionalidades dependen de que el cliente las utilicen de forma correcta lo que puede llegar a generar fallos	Competencia: Soluciones parciales fuertes como CoverManager, Revo o Glop.	Enfoque integral: El sistema busca centralizar múltiples funcionalidades en la misma página, esto aporta valor y diferenciación frente a otras opciones fragmentadas	Posibilidad de desarrollo modular: Se puede lanzar una versión inicial con funcionalidades clave (reservas, carta, reseñas) y luego ir ampliando.
	Riesgos de ciberseguridad al manejar datos personales de los consumidores.	Mejoras en la experiencia del cliente: Al hacer que el servicio sea más rápido, preciso y agradable y esto hace que los clientes tengan una mejor experiencia	Integración con plataformas existentes: Si se plantea como un sistema que puede conectarse con TPVs, apps de delivery o ERPs, se amplía su utilidad.
		Escalabilidad: El sistema es capaz de adaptarse a cualquier tipo de local	

Área	Horas estimadas	Coste por hora	Total
Planificación y diseño	130	40 €	5.200 €
Frontend	300	40 €	12.000 €
Backend	300	40 €	12.000 €
Base de datos	40	40 €	1.600 €
Testing y despliegue	100	40 €	4.000 €
TOTAL	870	— — —	(IVA Incluido) 34.800 €

“Al Punto” es una idea sólida, con potencial real en el mercado español y más allá. Tiene una propuesta de valor clara, pero necesita una fase de definición más profunda para evitar dispersión funcional y asegurar una ejecución viable. No es necesario replantearse, pero sí refinarlo, priorizar y validarlo antes de avanzar

PLANIFICACIÓN TEMPORAL

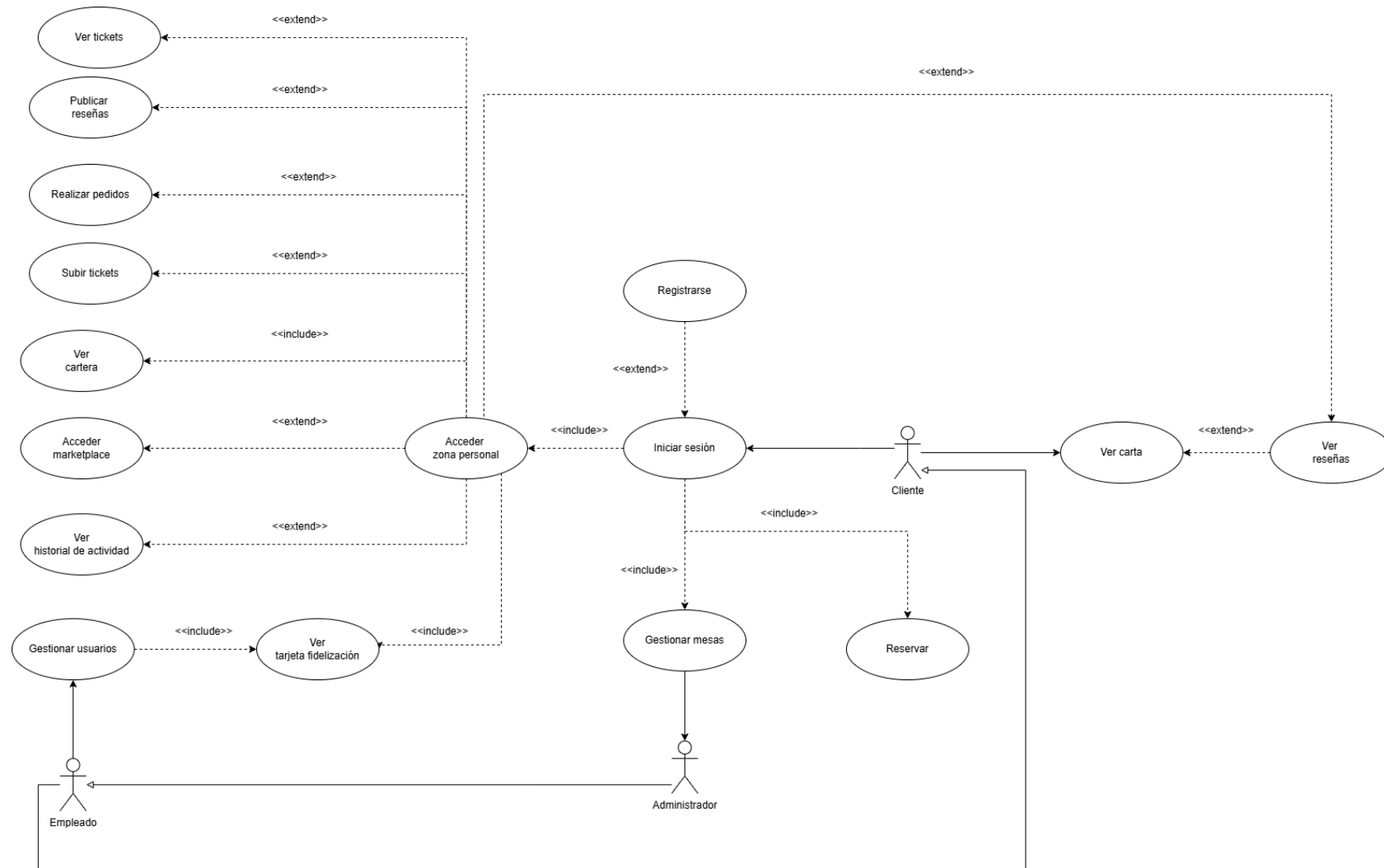
- **Hasta el 8 de enero**
 - Realización del Anteproyecto.
- **Del 8 al 30 de enero**
 - Hasta el 18 realizan casos de uso.
 - Desde el 19 al 30 realización del modelo entidad-relación.
- **Del 30 de enero al 13 de febrero**
 - Diagrama de clases y modelo de arquitectura.
- **Del 13 de febrero al 6 de marzo**
 - Primera implementación de la app.
- **Del 6 de marzo hasta el 29 de mayo**
 - Desarrollo y codificación completa de la aplicación.

4. Tecnologías escogidas y justificación

Tecnología	Área	Función
Node.js	Backend	Entorno de ejecución del servidor.
Express 5	Backend	Framework para la API REST.
SQLite	Backend	Base de datos relacional ligera.
JWT (JSON Web Tokens)	Backend	Sistema de autenticación.
BCrypt	Backend	Cifrado de contraseñas.
Vue 3	Frontend	Framework JavaScript para el frontend (SPA)
Vite	Frontend	Bundler y servidor de desarrollo

5. Diseño de la aplicación

5.1. Casos de Uso



CASO DE USO 1	Iniciar sesión
Actor	Cliente/Empleado/Admin
Precondición	El usuario tiene una cuenta registrada.
Flujo	1. El cliente accede a la pantalla de inicio de sesión. 2. Introduce usuario y contraseña. 3. El sistema valida las credenciales. 4. El sistema permite acceder a la Zona Personal.
Flujo alternativo	Si las credenciales son incorrectas, el sistema muestra un error. Si el usuario no tiene cuenta, puede acceder al registro.
Postcondición	El usuario queda autenticado con su rol asignado.

CASO DE USO 2	Registrarse
Actor	Cliente/Empleado/Admin
Precondición	El usuario no tiene cuenta en el sistema.
Flujo	1. El usuario accede al formulario de registro desde la pantalla de login. 2. Introduce nombre, correo y contraseña. 3. El sistema crea la cuenta y asigna el rol de Cliente por defecto. 4. Se confirma el registro.
Flujo alternativo	Si el correo ya está registrado, el sistema avisa y ofrece recuperar la contraseña.
Postcondición	El usuario tiene cuenta activa y puede iniciar sesión.

CASO DE USO 3	Acceder a "Zona personal"
Actor	Cliente/Empleado/Admin
Precondición	El usuario ha iniciado sesión correctamente.
Flujo	1. Tras el login, el sistema carga la SPA de zona personal. 2. Se muestran automáticamente la tarjeta de fidelización y el saldo 3. El usuario puede navegar al resto de funcionalidades.
Flujo alternativo	-
Postcondición	El usuario accede a su espacio personalizado con todas sus funcionalidades disponibles.

CASO DE USO 4	Ver tarjeta de fidelización
Actor	Cliente/Empleado/Admin

Precondición	El usuario está en la zona personal.
Flujo	1. Al cargar la zona personal, el sistema muestra automáticamente la tarjeta de fidelización digital con el código QR de identificación del usuario.
Flujo alternativo	-
Postcondición	El usuario visualiza su tarjeta con el código QR activo.

CASO DE USO 5	Ver cartera
Actor	Cliente/Empleado/Admin
Precondición	El usuario está en la zona personal.
Flujo	1. Al cargar la zona personal, el sistema muestra automáticamente el saldo de puntos acumulados y un gráfico de estadísticas con reservas, tickets y puntos.
Flujo alternativo	-
Postcondición	El usuario visualiza su saldo actualizado.

CASO DE USO 6	Ver tickets
Actor	Cliente/Empleado/Admin
Precondición	El usuario está en la zona personal y tiene tickets registrados.
Flujo	1. El usuario accede a la sección de tickets. 2. El sistema muestra el listado de tickets subidos previamente. 3. El usuario puede consultar el detalle de cada uno.
Flujo alternativo	-
Postcondición	El usuario visualiza sus tickets y los puntos asociados a cada uno.

CASO DE USO 7	Subir tickets
Actor	Cliente/Empleado/Admin
Precondición	El usuario está en la zona personal y tiene un ticket de consumición para subir.
Flujo	1. El usuario accede a la opción de subida de ticket. 2. Selecciona o fotografía el ticket. 3. El sistema procesa el ticket y calcula los puntos a acumular. 4. Los puntos se añaden al saldo del usuario.
Flujo alternativo	Si el ticket no es válido o ya fue subido, el sistema lo rechaza y notifica al usuario.
Postcondición	El ticket queda registrado y los puntos se reflejan en el saldo.

CASO DE USO 8	Publicar reseñas
Actor	Cliente/Empleado/Admin
Precondición	El usuario está en la zona personal.
Flujo	1. El usuario accede a la sección de comentarios. 2. Redacta su valoración sobre una consumición. 3. El sistema publica la reseña y la asocia al plato correspondiente.
Flujo alternativo	Si el usuario no ha consumido el plato, el sistema puede restringir la publicación.
Postcondición	La reseña queda publicada y visible en la carta del restaurante.

CASO DE USO 9	Publicar reseñas
Actor	Cliente/Empleado/Admin
Precondición	El usuario está en la zona personal.
Flujo	1. El usuario accede a la sección de comentarios. 2. Redacta su valoración sobre una consumición. 3. El sistema publica la reseña y la asocia al plato correspondiente.
Flujo alternativo	Si el usuario no ha consumido el plato, el sistema puede restringir la publicación.
Postcondición	La reseña queda publicada y visible en la carta del restaurante.

CASO DE USO 10	Realizar pedidos
Actor	Cliente/Empleado/Admin
Precondición	El usuario está en la zona personal.
Flujo	1. El usuario accede a la interfaz de carrito. 2. Selecciona los platos a consumir. 3. Confirma el pedido. 4. El sistema registra el pedido y notifica al restaurante.
Flujo alternativo	Si algún plato no está disponible, el sistema lo indica y permite eliminar el ítem.
Postcondición	El pedido queda registrado y en proceso de preparación.

CASO DE USO 11	Acceder marketplace
Actor	Cliente/Empleado/ Administrador
Precondición	El usuario está en la zona personal y tiene puntos acumulados.
Flujo	1. El usuario accede al marketplace. 2. El sistema muestra las ofertas canjeables según el saldo disponible. 3. El usuario selecciona una oferta y la canjea. 4. El sistema descuenta los puntos y registra el canje en el historial.
Flujo alternativo	Si el usuario no tiene suficientes puntos para una oferta, el sistema la muestra como no disponible.
Postcondición	El canje queda registrado y los puntos se descuentan del saldo.

CASO DE USO 12	Ver historial de actividad
Actor	Cliente/Empleado/ Administrador
Precondición	El usuario está en la zona personal.
Flujo	1. El usuario accede al historial. 2. El sistema muestra el registro cronológico de reservas, pedidos y ofertas del marketplace canjeadas. 3. El usuario puede filtrar por tipo de actividad.
Flujo alternativo	-
Postcondición	El usuario visualiza su actividad histórica completa.

CASO DE USO 13	Ver carta
Actor	Cliente/Empleado/ Administrador
Precondición	Ninguna, de acceso público.
Flujo	1. El usuario accede al menú interactivo del restaurante. 2. El sistema muestra los platos organizados por categorías con descripciones y precios.
Flujo alternativo	-
Postcondición	El usuario visualiza la carta completa del restaurante.

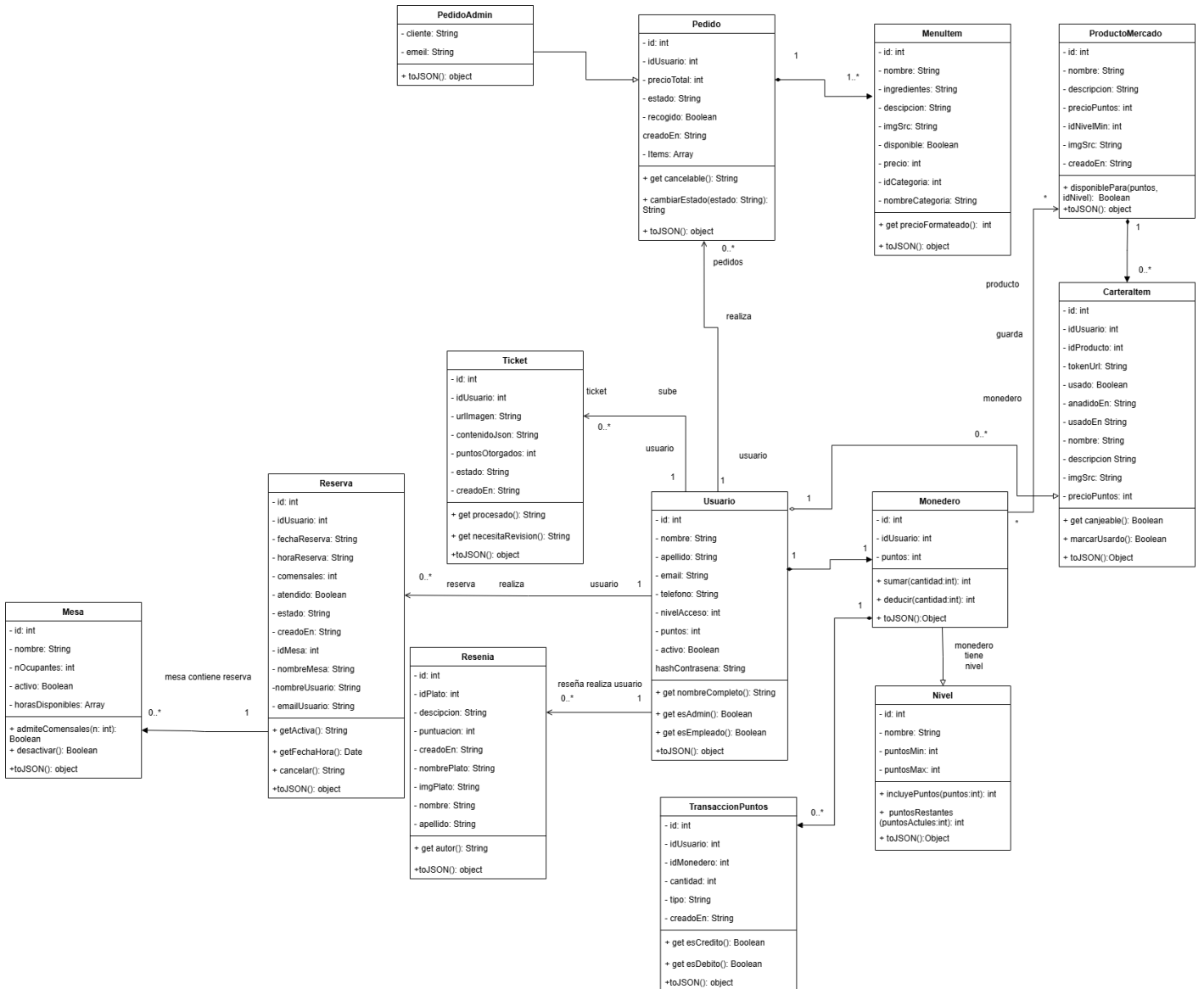
CASO DE USO 14	Ver reseñas
Actor	Cliente/Empleado/ Administrador
Precondición	El usuario está consultando la carta.
Flujo	1. El usuario selecciona un plato de la carta. 2. El sistema muestra las reseñas de otros usuarios asociadas a ese plato.
Flujo alternativo	-
Postcondición	El usuario visualiza las valoraciones del plato seleccionado.

CASO DE USO 15	Reservar
Actor	Cliente/Empleado/ Administrador
Precondición	El usuario ha iniciado sesión.
Flujo	1. El usuario accede al módulo de reservas. 2. Selecciona fecha, hora, número de comensales y mesa. 3. El sistema valida en tiempo real la disponibilidad. 4. El usuario confirma y el sistema registra la reserva.
Flujo alternativo	-
Postcondición	La reserva queda confirmada y registrada en el sistema.

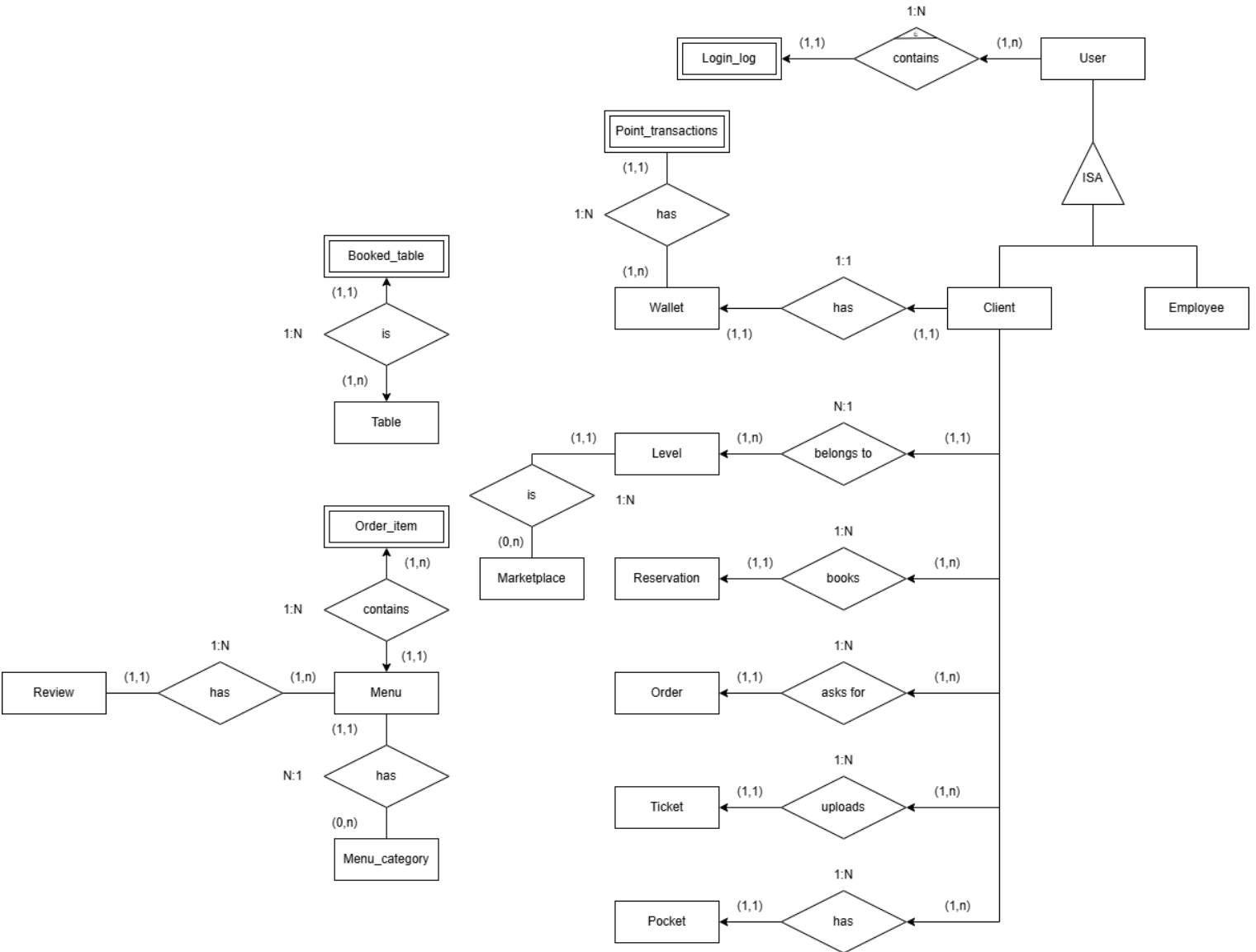
CASO DE USO 16	Gestionar usuarios
Actor	Empleado/Administrador
Precondición	El usuario ha iniciado sesión con rol de Empleado o superior.
Flujo	1. El empleado accede al panel de administración de usuarios. 2. Puede gestionar las reservas y pedidos pendientes, y canjear las ofertas del marketplace adquiridas por los usuarios. 3. El sistema registra los cambios realizados.
Flujo alternativo	-
Postcondición	Los cambios sobre los usuarios quedan persistidos en el sistema.

CASO DE USO 17	Gestionar mesas
Actor	Administrador
Precondición	El usuario ha iniciado sesión con rol de Administrador.
Flujo	1. El administrador accede al panel de gestión de mesas. 2. Puede crear, modificar o eliminar mesas del restaurante. 3. Configura la capacidad y disponibilidad de cada mesa. 4. El sistema actualiza la base de datos de disponibilidad en tiempo real.
Flujo alternativo	Si intenta eliminar una mesa con reservas activas, el sistema solicita confirmación.
Postcondición	La configuración de mesas queda actualizada y disponible para el módulo de reservas.

5.2. Diagrama de clases



5.3. Modelo Entidad-Relación y relacional





6. Arquitectura del proyecto

6.1. Estructura del proyecto

La aplicación está dividida en una arquitectura cliente-servidor, está compuesto por el frontend encargado de mostrar los datos al usuario y permitir la interacción y backend encargado de la gestión de los datos.

Arquitectura del Backend

El sistema de almacenamiento de datos se compone de una base de datos en SQLite la cual posee una autoiniciación partiendo de un archivo seed.sql.

La arquitectura del Backend se conforma por una arquitectura en capas, esta se compone de:

- routes: Entrada de las peticiones del front, define los endpoints de la API.
- controllers: Gestiona las peticiones que llegan desde el frontend a través de las rutas.
- services: Capa intermedia entre el controller y el repository se encarga de validar los datos.
- repository: Capa encargada de realizar las consultas a la base de datos.
- utils: Contiene diferentes herramientas, en este caso gestión de la base de datos, la encriptación de datos sensibles, control de tokens...
- uploads: Directorio que guarda las diferentes imágenes que se suben al back.
- classes: Define los objetos.

Arquitectura Frontend

La arquitectura del frontend es una arquitectura basada en componentes permitiendo una gran escalabilidad, este se organiza de esta manera:

- components: Aporta los componentes que se utilizan a lo largo de la aplicación.
- composables: Encargado de gestionar la sesión del usuario.
- router: Encargado de indicar las diferentes rutas de la página.
- services: Contiene los endpoints que realizan las peticiones al back.
- theme: Encargado de modificar en gran medida los colores de Ant Design.
- view: Contiene la estructura de páginas de la aplicación.

6.2 Librerías externas utilizadas

Backend:

- cors: gestión de políticas de acceso entre orígenes.
- dotenv: carga de variables de entorno.
- multer: gestión de subida de archivos.
- tesseract.js: OCR para extracción de texto en imágenes.
- nodemon: reinicio automático del servidor en desarrollo.

Frontend:

- axios: cliente HTTP para comunicación con la API.
- Ant Design Vue: librería de componentes UI.
- @antv/g2plot: visualización de datos y gráficas.
- crypto-js: cifrado en el lado del cliente.
- qrcode: generación de códigos QR.
- dayjs: manipulación y formateo de fechas.
- vue-router: enrutado de la SPA.
- vue-request: gestión de peticiones asíncronas.

7. Manual de despliegue

Requisitos mínimos de hardware y software:

- Hardware: Se recomienda que el dispositivo anfitrión cuente con un mínimo de 4 GB de RAM y 10 GB de almacenamiento libre.
- Navegadores Soportados: El correcto funcionamiento de la aplicación está garantizado en las últimas versiones (a fecha de 23/05/2026) de Google Chrome, Microsoft Edge y Brave. No se asegura el rendimiento óptimo en versiones anteriores o en otros navegadores.
- Dependencias del Servidor: Solo se requiere tener instalados Docker y el plugin de Docker Compose en el sistema anfitrión. No es necesario instalar Node.js ni configurar bases de datos locales, ya que toda la lógica de ejecución y dependencias quedan encapsuladas dentro de los contenedores.

Para la preparación del entorno se requieren dos ficheros “.env”, uno para el frontend y otro para el backend. Por ejemplo, en este último la ruta relativa sería ./backend/.env.

- Frontend: Contiene dos variables una la dirección del back y otra la clave para cifrar el id en la url del QR.
 - VITE_API_URL=http://localhost:5000
 - VITE_QR_SECRET=clave_secreta_compartida_con_backend
- Backend: contiene el puerto a dicha división y la clave de cifrado
 - PORT=5000
 - JWT_SECRET_KEY=EstoEsUnEjemploDeClaveUnicaParaEncriptarDeManeraSegura1234

Comenzaremos teniendo los dos directorios descargados para crear el primer archivo docker-compose.yml al mismo nivel que estos. Este se encargará de “orquestrar” la ejecución de los dos dockerfile, solamente se debe de tener en cuenta que para el correcto funcionamiento de la aplicación en el apartado de args en la línea 17 tanto en VITE_API_URL como en VITE_FRONTEND_URL se deberá indicar la dirección IP en el adaptador de red del equipo, para ello iremos al powershell usaremos el comando ipconfig nos desplazamos hasta el adaptador de Ethernet y seleccionaremos la IPv4, una vez identificado procederemos a incluirlas en el compose.

```
args:
- VITE_API_URL=TU_DIRECCION_IP:5000
- VITE_FRONTEND_URL=TU_DIRECCION_IP:5173
```

```
args:
- VITE_API_URL=http://192.168.1.66:5000
- VITE_FRONTEND_URL=http://192.168.1.66:5173
```

Docker-compose.yml

```
version: '3.8'

services:
  backend:
    image: alpunto/backend:latest
    build: ./backend
    ports:
      - "5000:5000"
    volumes:
      - ./backend/utils/db:/app/utils/db
      - ./backend/.env:/app/.env
    environment:
      - PORT=5000

  frontend:
    image: alpunto/frontend:latest
    build:
      context: ./frontend
    args:
      - VITE_API_URL=http://IP_PC:5000
      - VITE_FRONTEND_URL=http://IP_PC:5173
      - VITE_QR_SECRET=clave_secreta_compartida_con_backend
```

ports:

- "5173:5173"

Este será el formato del docker-compose, es de vital importancia tener cuidado a la hora de las tabulaciones.

Terminado el docker-compose seguiremos con la creación de los dos dockerfile, cada uno se colocará en su respectivo directorio.

Dockerfile del backend

```
FROM node:20-alpine

RUN apk add --no-cache \
    python3 \
    make \
    g++ \
    tesseract-ocr

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm rebuild sqlite3

EXPOSE 5000

CMD ["node", "server.js"]
```

Dockerfile del frontend

```
FROM node:20-alpine

WORKDIR /app
COPY package*.json ./
RUN npm install

COPY . .

ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL

ARG VITE_FRONTEND_URL
```

```
ENV VITE_FRONTEND_URL=$VITE_FRONTEND_URL
```

```
ARG VITE_QR_SECRET
```

```
ENV VITE_QR_SECRET=$VITE_QR_SECRET
```

```
RUN npm run build
```

```
RUN npm install -g serve
```

```
EXPOSE 5173
```

```
CMD ["serve", "-s", "dist", "-l", "5173", "--single"]
```

Una vez esté todo listo desde la carpeta inmediatamente superior a backend y frontend se descargan las imágenes del dockerhub: docker compose up -d. Dichas imágenes tienen como nombre alpunto/backend y alpunto/frontend.

Tras esto, se construye el contenedor con docker compose up --build y se accede desde Docker desktop a la página.

